

validate_balancing

July 4, 2026

1 Validación del Balanceo Demográfico por Dataset

Este notebook verifica que el script `2b-trainset_balancing.py` ha producido correctamente los datasets balanceados para cada fold. Para cada archivo balanceado disponible:

- Se compara con el dataset original filtrado al mismo fold
- Se comprueba que **solo el conjunto train ha sido modificado**
- Se verifica que `val` y `test` son bit-a-bit idénticos al original
- Se confirma que el efecto del balanceo es el esperado (más/menos filas, duplicados, sintéticos)
- Se revisa que no haya contaminación cruzada de información entre splits

Nota: Solo se analizan los archivos disponibles en `balanced_outputs/`. Los folds no presentes se omiten deliberadamente para este análisis de sanidad.

```
[1]: import pandas as pd
import numpy as np
from pathlib import Path
import re
from IPython.display import display, Markdown
import warnings
warnings.filterwarnings('ignore')

# Rutas
DATA_ROOT = Path(r'C:/Users/User/Desktop/TFM-Glucose-Prediction/data')

DATASETS = {
    'DIATREND': DATA_ROOT / 'DIATREND',
    'REPLACE-BG': DATA_ROOT / 'REPLACE-BG',
    'T1DiabetesGranada': DATA_ROOT / 'T1DiabetesGranada',
}

# Columnas de feature presentes en ambos archivos (para comparar val/test)
FEATURE_COLS = [f'x{i}' for i in range(8)] + ['y']
KEY_COLS = FEATURE_COLS + ['patient_id']

# Patrón de nombre del archivo balanceado
BAL_PATTERN = re.compile(
    r'.*_fold(?:<fold>\d+)?(?:<group>age|sex)?(?:<technique>.+)?\.parquet$'
)
```

```
print('Rutas configuradas correctamente.')
```

Rutas configuradas correctamente.

```
[2]: # Helpers

def load_original(dataset_dir: Path) -> pd.DataFrame:
    """Carga el archivo de ventanas original (con columnas fold_0..fold_4)."""
    candidates = sorted(dataset_dir.glob('windows_with_5folds_*.parquet'))
    if not candidates:
        raise FileNotFoundError(f'No se encontró archivo original en {dataset_dir}')
    return pd.read_parquet(candidates[0]), candidates[0].name

def get_original_split(df_orig: pd.DataFrame, fold_idx: int, split: str) -> pd.DataFrame:
    """Extrae las filas de un split concreto del archivo original para un fold dado."""
    col = f'fold_{fold_idx}'
    return df_orig[df_orig[col].str.lower() == split].reset_index(drop=True)

def get_patient_info(dataset_dir: Path) -> pd.DataFrame:
    for cand in ['Patient_info.parquet', 'patient_info.parquet', 'Patient_info.csv', 'patient_info.csv']:
        p = dataset_dir / cand
        if p.exists():
            return pd.read_parquet(p) if p.suffix == '.parquet' else pd.read_csv(p)
    raise FileNotFoundError(f'No patient_info en {dataset_dir}')

AGE_BINS = [-np.inf, 18, 30, 45, 60, np.inf]
AGE_LABELS = ['<=18', '19-30', '31-45', '46-60', '>60']

def add_age_group(df: pd.DataFrame, pid_col: str, pat_info: pd.DataFrame) -> pd.DataFrame:
    """Añade columna age_group al dataframe uniendo con patient_info."""
    pi = pat_info.copy()
    # Normalizar patient_id
    for c in pi.columns:
        if 'patient' in c.lower():
            pi['_pid'] = pi[c].astype(str).str.strip()
            break
    # Edad
```

```

for c in pi.columns:
    if c.lower() in ('age', 'birth_year'):
        if c.lower() == 'age':
            pi['_age'] = pd.to_numeric(pi[c], errors='coerce')
        else:
            pi['_age'] = 2026 - pd.to_numeric(pi[c], errors='coerce')
        break
pi['_age_group'] = pd.cut(pi['_age'], bins=AGE_BINS, labels=AGE_LABELS,
                          include_lowest=True).astype(str)
lookup = pi.set_index('_pid')['_age_group'].to_dict()
out = df.copy()
out['_age_group'] = df[pid_col].astype(str).str.strip().map(lookup).
↳ fillna('Unknown')
return out

def add_sex_group(df: pd.DataFrame, pid_col: str, pat_info: pd.DataFrame) -> pd.
↳ DataFrame:
    pi = pat_info.copy()
    for c in pi.columns:
        if 'patient' in c.lower():
            pi['_pid'] = pi[c].astype(str).str.strip()
            break
    sex_col = next((c for c in pi.columns if c.lower() == 'sex'), None)
    if sex_col is None:
        return df
    lookup = pi.set_index('_pid')[sex_col].str.upper().to_dict()
    out = df.copy()
    out['_sex_group'] = df[pid_col].astype(str).str.strip().map(lookup).
↳ fillna('Unknown')
    return out

def describe_distribution(df: pd.DataFrame, group_col: str, label: str) -> pd.
↳ DataFrame:
    counts = df[group_col].value_counts(dropna=False).sort_index()
    pct = (100 * counts / counts.sum()).round(1)
    result = pd.DataFrame({'n': counts, '%': pct})
    result.index.name = group_col
    result.columns = pd.MultiIndex.from_tuples([(label, 'n'), (label, '%')])
    return result

def check_val_test_integrity(orig_split: pd.DataFrame, bal_split: pd.DataFrame,
                             split_name: str) -> str:
    """Compara val o test del original vs balanceado; devuelve string de
    ↳ resultado."""
    if len(orig_split) != len(bal_split):

```

```

        return f' {split_name}: tamaños distintos ({len(orig_split)} vs_
↪{len(bal_split)})'

    cols = [c for c in KEY_COLS if c in orig_split.columns and c in bal_split.
↪columns]
    orig_sorted = orig_split[cols].sort_values(cols).reset_index(drop=True)
    bal_sorted = bal_split[cols].sort_values(cols).reset_index(drop=True)
    equal = orig_sorted.equals(bal_sorted)
    return f' {split_name}: idéntico al original ({len(orig_split)},) filas)'_
↪if equal \
        else f' {split_name}: mismo nº de filas pero contenido distinto -_
↪revisar'

def check_no_patient_overlap(train_df: pd.DataFrame,
                             val_df: pd.DataFrame,
                             test_df: pd.DataFrame) -> str:
    """Verifica que no hay pacientes comunes entre splits."""
    train_pids = set(train_df['patient_id'].astype(str))
    val_pids = set(val_df['patient_id'].astype(str))
    test_pids = set(test_df['patient_id'].astype(str))
    tv = train_pids & val_pids
    tt = train_pids & test_pids
    vt = val_pids & test_pids
    issues = []
    if tv: issues.append(f'train val={tv}')
    if tt: issues.append(f'train test={tt}')
    if vt: issues.append(f'val test={vt}')
    return ' Sin solapamiento de pacientes entre splits' if not issues \
        else ' SOLAPAMIENTO DE PACIENTES: ' + ' | '.join(issues)

def count_exact_duplicates(df: pd.DataFrame) -> int:
    """Número de filas que son duplicado exacto de otra en el mismo df."""
    cols = [c for c in FEATURE_COLS if c in df.columns]
    dup_mask = df[cols].duplicated(keep='first')
    return int(dup_mask.sum())

print('Helpers cargados.')
```

Helpers cargados.

1.1 Dataset 1: DIATREND

```
[3]: ds_name = 'DIATREND'
     ds_dir  = DATASETS[ds_name]

     df_orig_diatrend, orig_fname = load_original(ds_dir)
     pat_info_diatrend = get_patient_info(ds_dir)

     bal_files_diatrend = sorted((ds_dir / 'balanced_outputs').
                                ↪glob('windows_with_5folds_*.parquet'))
     # Excluir el archivo compacto si existe en balanced_outputs
     bal_files_diatrend = [f for f in bal_files_diatrend if BAL_PATTERN.match(f.
                                ↪name)]

     print(f'Original : {orig_fname} ({len(df_orig_diatrend):,} filas)')
     print(f'Balanceados disponibles: {len(bal_files_diatrend)}')
     for f in bal_files_diatrend:
         print(f' {f.name}')
```

Original : windows_with_5folds_DiaTrend_2026-03-27_PH4.parquet (2,304,689
filas)

Balanceados disponibles: 3

windows_with_5folds_DiaTrend_2026-03-27_PH4_fold0_age_oversampling.parquet
windows_with_5folds_DiaTrend_2026-03-27_PH4_fold1_sex_undersampling.parquet
windows_with_5folds_DiaTrend_2026-03-27_PH4_fold4_age_smote.parquet

1.1.1 DIATREND — Fold 0 · Age · Oversampling

Qué esperamos verificar: - El conjunto train del fold 0 debe tener **más filas** que en el original (se han añadido copias de ventanas de grupos minoritarios por edad). - Deben existir **duplicados exactos** en train, correspondientes a las ventanas replicadas. - Todos los grupos de edad deben haber alcanzado el tamaño del grupo mayoritario (el target es `max(n_grupo)`). - Los conjuntos val y test del fold 0 deben ser **bit-a-bit idénticos** al original. - Ningún paciente debe aparecer en más de un split.

```
[4]: bal_file = ds_dir / 'balanced_outputs' /_
     ↪'windows_with_5folds_DiaTrend_2026-03-27_PH4_fold0_age_oversampling.parquet'
     df_bal    = pd.read_parquet(bal_file)

     fold_idx = 0
     group     = 'age'

     # Extraer splits
     bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
     bal_val    = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
     bal_test   = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

     orig_train = get_original_split(df_orig_diatrend, fold_idx, 'train')
```

```

orig_val = get_original_split(df_orig_diatrend, fold_idx, 'val')
orig_test = get_original_split(df_orig_diatrend, fold_idx, 'test')

# Añadir grupos demográficos para comparar distribución
orig_train_g = add_age_group(orig_train, 'patient_id', pat_info_diatrend)
bal_train_g = add_age_group(bal_train, 'patient_id', pat_info_diatrend)

# 1. Tamaños de train
print(' 1. TAMAÑO DE TRAIN')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
print(f' Δ : {delta:>+10,} filas {"(↑ oversampling aplicado )" if_
    ↪delta > 0 else "( no aumentó)"}')

# 2. Distribución por grupo de edad
print('\n 2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN')
dist_orig = describe_distribution(orig_train_g, 'age_group', 'Original')
dist_bal = describe_distribution(bal_train_g, 'age_group', 'Balanceado')
display(pd.concat([dist_orig, dist_bal], axis=1).fillna(0))

# 3. Duplicados exactos en train
n_dup = count_exact_duplicates(bal_train)
print(f'\n 3. DUPLICADOS EXACTOS EN TRAIN (oversampling)')
print(f' Filas duplicadas: {n_dup:,} {" Existen duplicados - oversampling_
    ↪confirmado" if n_dup > 0 else " Sin duplicados"}')

# 4. Integridad de val y test
print('\n 4. INTEGRIDAD DE VAL Y TEST (no deben haberse modificado)')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

# 5. No solapamiento de pacientes
print('\n 5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

1. TAMAÑO DE TRAIN

Original : 1,598,896 filas

Balanceado: 6,056,508 filas

Δ : +4,457,612 filas (↑ oversampling aplicado)

2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN

	Original		Balanceado	
	n	%	n	%
age_group				
19-30	1514127	94.7	1514127	25.0
31-45	20144	1.3	1514127	25.0

46-60	48725	3.0	1514127	25.0
>60	15900	1.0	1514127	25.0

3. DUPLICADOS EXACTOS EN TRAIN (oversampling)

Filas duplicadas: 4,459,496 Existen duplicados - oversampling confirmado

4. INTEGRIDAD DE VAL Y TEST (no deben haberse modificado)

val: idéntico al original (262,079 filas)

test: idéntico al original (443,714 filas)

5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.1.2 DIATREND — Fold 1 · Sex · Undersampling

Qué esperamos verificar: - El conjunto train del fold 1 debe tener **menos filas** que en el original (el grupo mayoritario por sexo se ha reducido al tamaño del minoritario). - **No debe haber duplicados exactos** en train (undersampling nunca añade filas). - Ambos grupos de sexo deben tener el mismo número de ventanas. - val y test del fold 1 intactos.

```
[5]: bal_file = ds_dir / 'balanced_outputs' / \
    ↪ 'windows_with_5folds_DiaTrend_2026-03-27_PH4_fold1_sex_undersampling.parquet'
df_bal = pd.read_parquet(bal_file)

fold_idx = 1
group = 'sex'

bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
bal_val = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
bal_test = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

orig_train = get_original_split(df_orig_diatrend, fold_idx, 'train')
orig_val = get_original_split(df_orig_diatrend, fold_idx, 'val')
orig_test = get_original_split(df_orig_diatrend, fold_idx, 'test')

orig_train_g = add_sex_group(orig_train, 'patient_id', pat_info_diatrend)
bal_train_g = add_sex_group(bal_train, 'patient_id', pat_info_diatrend)

print(' 1. TAMAÑO DE TRAIN ')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
print(f' Δ : {delta:>+10,} filas {"(↓ undersampling aplicado )" if \
    ↪ delta < 0 else "( no redujo)"}')

print('\n 2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN ')
dist_orig = describe_distribution(orig_train_g, 'sex_group', 'Original')
```

```

dist_bal = describe_distribution(bal_train_g, 'sex_group', 'Balanceado')
display(pd.concat([dist_orig, dist_bal], axis=1).fillna(0))

n_dup = count_exact_duplicates(bal_train)
print(f'\n 3. DUPLICADOS EXACTOS EN TRAIN (undersampling no crea      ')
print(f' Filas duplicadas: {n_dup:,} {" Sin duplicados - undersampling_
↳confirmado" if n_dup == 0 else " Hay duplicados - revisar"}')

print('\n 4. INTEGRIDAD DE VAL Y TEST                                ')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS                ')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

1. TAMAÑO DE TRAIN

Original : 1,613,241 filas

Balanceado: 484,770 filas

Δ : -1,128,471 filas (↓ undersampling aplicado)

2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN

	Original		Balanceado	
sex_group	n	%	n	%
F	1370856	85.0	242385	50.0
M	242385	15.0	242385	50.0

3. DUPLICADOS EXACTOS EN TRAIN (undersampling no crea)

Filas duplicadas: 319 Hay duplicados - revisar

4. INTEGRIDAD DE VAL Y TEST

val: idéntico al original (216,716 filas)

test: idéntico al original (474,732 filas)

5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.1.3 DIATREND — Fold 4 · Age · SMOTE

Qué esperamos verificar: - El conjunto train del fold 4 debe tener **más filas** que el original (se han generado ventanas sintéticas para los grupos minoritarios). - Las ventanas generadas por SMOTE son **interpolaciones** de ventanas reales, por lo que **no deberían ser duplicados exactos** de ninguna fila del train original. - Los valores de glucosa de las ventanas sintéticas deben estar dentro de los límites del sensor (DiaTrend: [39, 401] mg/dL). - val y test del fold 4 intactos.


```

[6]: bal_file = ds_dir / 'balanced_outputs' /
      ↪ 'windows_with_5folds_DiaTrend_2026-03-27_PH4_fold4_age_smote.parquet'
df_bal = pd.read_parquet(bal_file)

fold_idx = 4
group = 'age'
SENSOR_MIN, SENSOR_MAX = 39.0, 401.0

bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
bal_val = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
bal_test = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

orig_train = get_original_split(df_orig_diatrend, fold_idx, 'train')
orig_val = get_original_split(df_orig_diatrend, fold_idx, 'val')
orig_test = get_original_split(df_orig_diatrend, fold_idx, 'test')

orig_train_g = add_age_group(orig_train, 'patient_id', pat_info_diatrend)
bal_train_g = add_age_group(bal_train, 'patient_id', pat_info_diatrend)

print(' 1. TAMAÑO DE TRAIN ')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
print(f' Δ : {delta:>+10,} filas {"(↑ SMOTE generó sintéticos )" if
      ↪ delta > 0 else "( no aumentó)"}')

print('\n 2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN ')
dist_orig = describe_distribution(orig_train_g, 'age_group', 'Original')
dist_bal = describe_distribution(bal_train_g, 'age_group', 'Balanceado')
display(pd.concat([dist_orig, dist_bal], axis=1).fillna(0))

# Duplicados exactos: SMOTE no debe crear copias exactas (son interpolaciones)
n_dup = count_exact_duplicates(bal_train)
print(f'\n 3. DUPLICADOS EXACTOS EN TRAIN (SMOTE genera puntos interpolados)
      ↪ ')
print(f' Filas duplicadas: {n_dup:,} {" Muy pocos/ningún duplicado - SMOTE
      ↪ confirmado" if n_dup < 10 else " Muchos duplicados - revisar"}')

# Cuántas filas de train_bal no están en orig_train (son nuevas/sintéticas)
feat_cols = [c for c in FEATURE_COLS if c in bal_train.columns]
orig_set = set(map(tuple, orig_train[feat_cols].round(4).values.tolist()))
synthetic_mask = ~bal_train[feat_cols].round(4).apply(tuple, axis=1).
      ↪ isin(orig_set)
n_synthetic = synthetic_mask.sum()
print(f' Filas sintéticas (no presentes en original): {n_synthetic:,}')

# Comprobar límites del sensor en las filas sintéticas

```

```

if n_synthetic > 0:
    synth_df = bal_train[synthetic_mask][feat_cols]
    out_of_range = ((synth_df < SENSOR_MIN) | (synth_df > SENSOR_MAX)).
    ↪any(axis=1).sum()
    print(f' Sintéticos fuera de rango [{SENSOR_MIN}, {SENSOR_MAX}]:_
    ↪{out_of_range} '
        f'{" Todos dentro del rango" if out_of_range == 0 else " Hay valores_
    ↪fuera de rango"}')

print('\n 4. INTEGRIDAD DE VAL Y TEST ')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS ')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

1. TAMAÑO DE TRAIN

Original : 1,604,177 filas

Balanceado: 4,652,544 filas

Δ : +3,048,367 filas (↑ SMOTE generó sintéticos)

2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN

	Original		Balanceado	
age_group	n	%	n	%
19-30	1550848	96.7	1550848	33.3
31-45	28214	1.8	1550848	33.3
46-60	25115	1.6	1550848	33.3

3. DUPLICADOS EXACTOS EN TRAIN (SMOTE genera puntos interpolados)

Filas duplicadas: 1,496 Muchos duplicados - revisar

Filas sintéticas (no presentes en original): 3,048,188

Sintéticos fuera de rango [39.0, 401.0]: 0 Todos dentro del rango

4. INTEGRIDAD DE VAL Y TEST

val: idéntico al original (223,944 filas)

test: idéntico al original (476,568 filas)

5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.2 Dataset 2: REPLACE-BG

```
[7]: ds_name = 'REPLACE-BG'
     ds_dir  = DATASETS[ds_name]

     df_orig_replace, orig_fname = load_original(ds_dir)
     pat_info_replace = get_patient_info(ds_dir)

     bal_files_replace = sorted((ds_dir / 'balanced_outputs').
                                ↪glob('windows_with_5folds_*.parquet'))
     bal_files_replace = [f for f in bal_files_replace if BAL_PATTERN.match(f.name)]

     print(f'Original : {orig_fname} ({len(df_orig_replace):,} filas)')
     print(f'Balanceados disponibles: {len(bal_files_replace)}')
     for f in bal_files_replace:
         print(f' {f.name}')
```

```
Original : windows_with_5folds_REPLACE-BG_2026-05-26_PH4.parquet (3,305,411
filas)
Balanceados disponibles: 3
  windows_with_5folds_REPLACE-
BG_2026-05-26_PH4_fold0_age_reference_proportional.parquet
  windows_with_5folds_REPLACE-
BG_2026-05-26_PH4_fold1_sex_patient_aware_undersampling.parquet
  windows_with_5folds_REPLACE-BG_2026-05-26_PH4_fold2_age_undersampling.parquet
```

1.2.1 REPLACE-BG — Fold 0 · Age · Reference Proportional

Qué esperamos verificar: - El train del fold 0 debe tener una distribución de grupos de edad **distinta** a la original, alineada con las proporciones del archivo de referencia externo. - Algunos grupos habrán aumentado (oversampling interno) y otros habrán disminuido (undersampling interno). - El **número total de filas de train** debe ser similar al original (se preserva el volumen). - val y test intactos.

```
[8]: bal_file = ds_dir / 'balanced_outputs' /
     ↪'windows_with_5folds_REPLACE-BG_2026-05-26_PH4_fold0_age_reference_proportional.
     ↪parquet'
     df_bal = pd.read_parquet(bal_file)

     # Intentar cargar el archivo de referencia si existe
     ref_file = DATA_ROOT / 'reference_distributions' / 'age_reference_gregory2022.
     ↪json'
     ref_props = None
     if ref_file.exists():
         import json
         with open(ref_file) as f:
             ref_props = json.load(f)
         print('Proporciones de referencia cargadas:')
```

```

    for k, v in sorted(ref_props.items()):
        print(f' {k}: {100*v:.1f}%')
else:
    print('(Archivo de referencia no disponible en esta ruta)')

fold_idx = 0

bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
bal_val    = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
bal_test   = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

orig_train = get_original_split(df_orig_replace, fold_idx, 'train')
orig_val    = get_original_split(df_orig_replace, fold_idx, 'val')
orig_test   = get_original_split(df_orig_replace, fold_idx, 'test')

orig_train_g = add_age_group(orig_train, 'patient_id', pat_info_replace)
bal_train_g   = add_age_group(bal_train, 'patient_id', pat_info_replace)

print('\n 1. TAMAÑO DE TRAIN')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
pct_change = 100 * delta / len(orig_train) if len(orig_train) > 0 else 0
print(f' Δ : {delta:>+10,} filas ({pct_change:+.1f}%) (volumen_
↳aproximadamente conservado si cercano a 0)')

print('\n 2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN')
dist_orig = describe_distribution(orig_train_g, 'age_group', 'Original')
dist_bal   = describe_distribution(bal_train_g, 'age_group', 'Balanceado')
combined   = pd.concat([dist_orig, dist_bal], axis=1).fillna(0)
if ref_props:
    ref_series = pd.Series({k: round(100*v, 1) for k, v in ref_props.items()},
↳name=('Referencia', '%'))
    combined   = pd.concat([combined, ref_series], axis=1)
display(combined)

print('\n 3. INTEGRIDAD DE VAL Y TEST')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 4. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

Proporciones de referencia cargadas:

19-30: 17.0%
31-45: 24.0%
46-60: 30.0%

<=18: 8.0%
>60: 21.0%

1. TAMAÑO DE TRAIN

Original : 2,308,739 filas

Balanceado: 2,308,739 filas

Δ : +0 filas (+0.0%) (volumen aproximadamente conservado si cercano a 0)

2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN

	Original		Balanceado		Referencia
	n	%	n	%	%
19-30	519260.0	22.5	426615.0	18.5	17.0
31-45	687000.0	29.8	602280.0	26.1	24.0
46-60	791087.0	34.3	752849.0	32.6	30.0
>60	311392.0	13.5	526995.0	22.8	21.0
<=18	NaN	NaN	NaN	NaN	8.0

3. INTEGRIDAD DE VAL Y TEST

val: idéntico al original (335,738 filas)

test: idéntico al original (660,934 filas)

4. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.2.2 REPLACE-BG — Fold 1 · Sex · Patient-Aware Undersampling

Qué esperamos verificar: - El train del fold 1 debe tener **menos filas** que el original. - La reducción debe distribuirse proporcionalmente entre los pacientes del grupo mayoritario, sin eliminar a ningún paciente completamente (el patient-aware undersampling pone un cap por paciente). - Ambos grupos de sexo deben quedar equilibrados. - val y test intactos.

```
[9]: bal_file = ds_dir / 'balanced_outputs' / \
    ↪ 'windows_with_5folds_REPLACE-BG_2026-05-26_PH4_fold1_sex_patient_aware_undersampling.'
    ↪ 'parquet'
df_bal = pd.read_parquet(bal_file)

fold_idx = 1

bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
bal_val = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
bal_test = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

orig_train = get_original_split(df_orig_replace, fold_idx, 'train')
orig_val = get_original_split(df_orig_replace, fold_idx, 'val')
orig_test = get_original_split(df_orig_replace, fold_idx, 'test')
```

```

orig_train_g = add_sex_group(orig_train, 'patient_id', pat_info_replace)
bal_train_g  = add_sex_group(bal_train,  'patient_id', pat_info_replace)

print(' 1. TAMAÑO DE TRAIN')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
print(f' Δ : {delta:>+10,} filas  {(↓ undersampling aplicado )" if Δ
    ↪ delta < 0 else "( no redujo)"})')

print('\n 2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN')
dist_orig = describe_distribution(orig_train_g, 'sex_group', 'Original')
dist_bal  = describe_distribution(bal_train_g,  'sex_group', 'Balanceado')
display(pd.concat([dist_orig, dist_bal], axis=1).fillna(0))

# Verificar que no se eliminaron pacientes enteros (patient-aware)
orig_pids_per_group = orig_train_g.groupby('sex_group')['patient_id'].nunique()
bal_pids_per_group  = bal_train_g.groupby('sex_group')['patient_id'].nunique()
print('\n 3. PACIENTES CONSERVADOS POR GRUPO (patient-aware no elimina
    ↪pacientes enteros) ')
comparison = pd.DataFrame({'Original': orig_pids_per_group, 'Balanceado':
    ↪bal_pids_per_group})
display(comparison)
all_conserved = (comparison['Balanceado'] == comparison['Original']).all()
print(f' {' Todos los pacientes conservados' if all_conserved else ' Algún
    ↪paciente fue eliminado completamente'})

n_dup = count_exact_duplicates(bal_train)
print(f'\n 4. DUPLICADOS (undersampling no crea)')
print(f' Filas duplicadas: {n_dup:,} {" Sin duplicados" if n_dup == 0 else "
    ↪ Hay duplicados"}')

print('\n 5. INTEGRIDAD DE VAL Y TEST')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 6. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

1. TAMAÑO DE TRAIN

Original : 2,309,304 filas

Balanceado: 2,267,201 filas

Δ : -42,103 filas (↓ undersampling aplicado)

2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN

Original	Balanceado
----------	------------

	n	%	n	%
sex_group				
F	1151769	49.9	1151769	50.8
M	1157535	50.1	1115432	49.2

3. PACIENTES CONSERVADOS POR GRUPO (patient-aware no elimina pacientes enteros)

	Original	Balanceado
sex_group		
F	79	79
M	76	76

Todos los pacientes conservados

4. DUPLICADOS (undersampling no crea)
Filas duplicadas: 900 Hay duplicados

5. INTEGRIDAD DE VAL Y TEST
val: idéntico al original (334,683 filas)
test: idéntico al original (661,424 filas)

6. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS
Sin solapamiento de pacientes entre splits

1.2.3 REPLACE-BG — Fold 2 · Age · Undersampling

Qué esperamos verificar: - Train reducido, sin duplicados, ambos grupos al mismo nivel (mínimo del original). - val y test del fold 2 intactos.

```
[10]: bal_file = ds_dir / 'balanced_outputs' / \
      ↪ 'windows_with_5folds_REPLACE-BG_2026-05-26_PH4_fold2_age_undersampling.'
      ↪ 'parquet'
df_bal = pd.read_parquet(bal_file)

fold_idx = 2

bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
bal_val = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
bal_test = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

orig_train = get_original_split(df_orig_replace, fold_idx, 'train')
orig_val = get_original_split(df_orig_replace, fold_idx, 'val')
orig_test = get_original_split(df_orig_replace, fold_idx, 'test')

orig_train_g = add_age_group(orig_train, 'patient_id', pat_info_replace)
bal_train_g = add_age_group(bal_train, 'patient_id', pat_info_replace)
```

```

print(' 1. TAMAÑO DE TRAIN')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
print(f' Δ : {delta:>+10,} filas {"(↓ undersampling )" if delta < 0
↪ else "( no redujo)"}')

print('\n 2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN')
dist_orig = describe_distribution(orig_train_g, 'age_group', 'Original')
dist_bal = describe_distribution(bal_train_g, 'age_group', 'Balanceado')
display(pd.concat([dist_orig, dist_bal], axis=1).fillna(0))

# El target de undersampling es el mínimo del original
orig_counts = orig_train_g['age_group'].value_counts()
expected_target = int(orig_counts.min())
print(f' Target esperado (min grupo original): {expected_target:,}')
bal_counts = bal_train_g['age_group'].value_counts()
for g, n in bal_counts.items():
    ok = ' ' if n <= expected_target else ' '
    print(f' {g}: {n:,} {ok}')

n_dup = count_exact_duplicates(bal_train)
print(f'\n 3. DUPLICADOS')
print(f' Filas duplicadas: {n_dup:,} {" Sin duplicados" if n_dup == 0 else "
↪ Hay duplicados"}')

print('\n 4. INTEGRIDAD DE VAL Y TEST')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

1. TAMAÑO DE TRAIN

Original : 2,313,695 filas

Balanceado: 1,277,544 filas

Δ : -1,036,151 filas (↓ undersampling)

2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN

	Original		Balanceado	
	n	%	n	%
age_group				
19-30	489193	21.1	319386	25.0
31-45	660555	28.5	319386	25.0
46-60	844561	36.5	319386	25.0
>60	319386	13.8	319386	25.0

Target esperado (min grupo original): 319,386
31-45: 319,386
19-30: 319,386
46-60: 319,386
>60: 319,386

3. DUPLICADOS

Filas duplicadas: 405 Hay duplicados

4. INTEGRIDAD DE VAL Y TEST

val: idéntico al original (330,135 filas)
test: idéntico al original (661,581 filas)

5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.3 Dataset 3: T1DiabetesGranada

```
[11]: ds_name = 'T1DiabetesGranada'
      ds_dir = DATASETS[ds_name]

      df_orig_t1dg, orig_fname = load_original(ds_dir)
      # patient_info en CSV para T1DG
      pat_info_t1dg = pd.read_csv(ds_dir / 'Patient_info.csv') if (ds_dir /
      ↪ 'Patient_info.csv').exists() \
      else get_patient_info(ds_dir)

      bal_files_t1dg = sorted((ds_dir / 'balanced_outputs').
      ↪ glob('windows_with_5folds*.parquet'))
      bal_files_t1dg = [f for f in bal_files_t1dg if BAL_PATTERN.match(f.name)]

      print(f'Original : {orig_fname} ({len(df_orig_t1dg):,} filas)')
      print(f'Balanceados disponibles: {len(bal_files_t1dg)}')
      for f in bal_files_t1dg:
          print(f' {f.name}')
```

Original : windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4.parquet
(19,421,406 filas)

Balanceados disponibles: 3

 windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4_fold4_age_smote.parquet
 windows_with_5folds_T1DiabetesGranada_2026-05-
26_PH4_fold4_sex_oversampling.parquet
 windows_with_5folds_T1DiabetesGranada_2026-05-
26_PH4_fold4_sex_undersampling.parquet

Qué esperamos verificar: - Más filas en train, ventanas sintéticas interpoladas, sin duplicados exactos, dentro de rango [40, 500] mg/dL (límites de T1DG). - **val** y **test** del fold 4 intactos.

18

```

synth_df = bal_train[synthetic_mask][feat_cols]
out_of_range = ((synth_df < SENSOR_MIN) | (synth_df > SENSOR_MAX)).
↳any(axis=1).sum()
print(f' Sintéticos fuera de rango [{SENSOR_MIN}, {SENSOR_MAX}]:↳
↳{out_of_range} '
      f'{" " if out_of_range == 0 else " "}')
print(f' Rango observado - min: {synth_df.min().min():.2f} max: {synth_df.
↳max().max():.2f}')

print('\n 4. INTEGRIDAD DE VAL Y TEST ')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS ')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

1. TAMAÑO DE TRAIN

Original : 13,583,812 filas

Balanceado: 21,499,470 filas

Δ : +7,915,658 filas (↑ SMOTE)

2. DISTRIBUCIÓN POR AGE_GROUP EN TRAIN

	Original		Balanceado	
age_group	n	%	n	%
19-30	2589517	19.1	7262457	33.8
31-45	3918923	28.8	4453362	20.7
46-60	4511256	33.2	4596279	21.4
>60	2564116	18.9	5187372	24.1

3. SINTÉTICOS VS DUPLICADOS

Filas sintéticas (nuevas): 7,901,842

Duplicados exactos : 25,304

Sintéticos fuera de rango [40.0, 500.0]: 0

Rango observado - min: 40.00 max: 500.00

4. INTEGRIDAD DE VAL Y TEST

val: idéntico al original (1,943,190 filas)

test: idéntico al original (3,894,404 filas)

5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.3.2 T1DiabetesGranada — Fold 4 · Sex · Oversampling

Qué esperamos verificar: - Más filas en train, con duplicados exactos del grupo minoritario por sexo. - Ambos sexos al nivel del grupo mayoritario. - val y test del fold 4 idénticos al original.

```

[13]: bal_file = ds_dir / 'balanced_outputs' / \
      ↪ 'windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4_fold4_sex_oversampling.'
      ↪ 'parquet'
df_bal = pd.read_parquet(bal_file)

fold_idx = 4

bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
bal_val = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
bal_test = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

orig_train = get_original_split(df_orig_t1dg, fold_idx, 'train')
orig_val = get_original_split(df_orig_t1dg, fold_idx, 'val')
orig_test = get_original_split(df_orig_t1dg, fold_idx, 'test')

orig_train_g = add_sex_group(orig_train, 'patient_id', pat_info_t1dg)
bal_train_g = add_sex_group(bal_train, 'patient_id', pat_info_t1dg)

print(' 1. TAMAÑO DE TRAIN')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
print(f' Δ : {delta:>+10,} filas {"(↑ oversampling )" if delta > 0
      ↪ else "( no aumentó)"}')

print('\n 2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN')
dist_orig = describe_distribution(orig_train_g, 'sex_group', 'Original')
dist_bal = describe_distribution(bal_train_g, 'sex_group', 'Balanceado')
display(pd.concat([dist_orig, dist_bal], axis=1).fillna(0))

n_dup = count_exact_duplicates(bal_train)
print(f'\n 3. DUPLICADOS EXACTOS (oversampling los introduce)')
print(f' Filas duplicadas: {n_dup:,} {" Duplicados presentes - oversampling_
      ↪ confirmado" if n_dup > 0 else " Sin duplicados"}')

print('\n 4. INTEGRIDAD DE VAL Y TEST')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

```

1. TAMAÑO DE TRAIN
Original : 13,583,812 filas
Balanceado: 14,818,976 filas
Δ : +1,235,164 filas (↑ oversampling )

```

2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN

	Original		Balanceado	
	n	%	n	%
sex_group				
F	7409488	54.5	7409488	50.0
M	6174324	45.5	7409488	50.0

3. DUPLICADOS EXACTOS (oversampling los introduce)

Filas duplicadas: 3,103,782 Duplicados presentes - oversampling confirmado

4. INTEGRIDAD DE VAL Y TEST

val: idéntico al original (1,943,190 filas)

test: idéntico al original (3,894,404 filas)

5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.3.3 T1DiabetesGranada — Fold 4 · Sex · Undersampling

Qué esperamos verificar: - Train reducido. Sin duplicados. Ambos sexos igualados al mínimo. - val y test del fold 4 intactos (mismos que en el oversampling anterior, ya que el fold es el mismo).

```
[14]: bal_file = ds_dir / 'balanced_outputs' / \
      ↪ 'windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4_fold4_sex_undersampling.'
      ↪ 'parquet'
df_bal = pd.read_parquet(bal_file)

fold_idx = 4

bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
bal_val = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
bal_test = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)

orig_train = get_original_split(df_orig_t1dg, fold_idx, 'train')
orig_val = get_original_split(df_orig_t1dg, fold_idx, 'val')
orig_test = get_original_split(df_orig_t1dg, fold_idx, 'test')

orig_train_g = add_sex_group(orig_train, 'patient_id', pat_info_t1dg)
bal_train_g = add_sex_group(bal_train, 'patient_id', pat_info_t1dg)

print(' 1. TAMAÑO DE TRAIN ')
print(f' Original : {len(orig_train):>10,} filas')
print(f' Balanceado: {len(bal_train):>9,} filas')
delta = len(bal_train) - len(orig_train)
print(f' Δ : {delta:>+10,} filas {"(↓ undersampling )" if delta < 0
      ↪ else "( no redujo)"}')
```

```

print('\n 2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN')
dist_orig = describe_distribution(orig_train_g, 'sex_group', 'Original')
dist_bal = describe_distribution(bal_train_g, 'sex_group', 'Balanceado')
display(pd.concat([dist_orig, dist_bal], axis=1).fillna(0))

orig_counts = orig_train_g['sex_group'].value_counts()
expected_target = int(orig_counts.min())
bal_counts = bal_train_g['sex_group'].value_counts()
print(f' Target esperado (min grupo original): {expected_target:,}')
for g, n in bal_counts.items():
    ok = ' ' if n <= expected_target else ' '
    print(f' {g}: {n:,} {ok}')

n_dup = count_exact_duplicates(bal_train)
print(f'\n 3. DUPLICADOS')
print(f' Filas duplicadas: {n_dup:,} {" Sin duplicados" if n_dup == 0 else " Hay duplicados"}')

# Bonus: comparar val/test con el archivo de oversampling del mismo fold
# (ambos deben tener exactamente las mismas filas de val y test)
bal_over_file = ds_dir / 'balanced_outputs' /
    'windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4_fold4_sex_oversampling.
    parquet'
if bal_over_file.exists():
    df_over = pd.read_parquet(bal_over_file)
    over_val = df_over[df_over['split'] == 'val'].reset_index(drop=True)
    over_test = df_over[df_over['split'] == 'test'].reset_index(drop=True)
    print('\n BONUS: val/test idénticos entre técnicas del mismo fold')
    print(' ', check_val_test_integrity(over_val, bal_val, 'val (vs
    oversampling)'))
    print(' ', check_val_test_integrity(over_test, bal_test, 'test (vs
    oversampling)'))

print('\n 4. INTEGRIDAD DE VAL Y TEST')
print(' ', check_val_test_integrity(orig_val, bal_val, 'val'))
print(' ', check_val_test_integrity(orig_test, bal_test, 'test'))

print('\n 5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS')
print(' ', check_no_patient_overlap(bal_train, bal_val, bal_test))

```

1. TAMAÑO DE TRAIN

Original : 13,583,812 filas

Balanceado: 12,348,648 filas

Δ : -1,235,164 filas (↓ undersampling)

2. DISTRIBUCIÓN POR SEX_GROUP EN TRAIN

Original	Balanceado
----------	------------

	n	%	n	%
sex_group				
F	7409488	54.5	6174324	50.0
M	6174324	45.5	6174324	50.0

Target esperado (min grupo original): 6,174,324

F: 6,174,324

M: 6,174,324

3. DUPLICADOS

Filas duplicadas: 10,558 Hay duplicados

BONUS: val/test idénticos entre técnicas del mismo fold

val (vs oversampling): idéntico al original (1,943,190 filas)

test (vs oversampling): idéntico al original (3,894,404 filas)

4. INTEGRIDAD DE VAL Y TEST

val: idéntico al original (1,943,190 filas)

test: idéntico al original (3,894,404 filas)

5. SOLAPAMIENTO DE PACIENTES ENTRE SPLITS

Sin solapamiento de pacientes entre splits

1.4 Resumen global

La celda siguiente agrega todos los resultados en una tabla de validación para tener una vista de conjunto.

```
[15]: EXPERIMENTS = [
    ('DIATREND',          df_orig_diatrend,  pat_info_diatrend,
     'windows_with_5folds_DiaTrend_2026-03-27_PH4_fold0_age_oversampling.
↪parquet',              0, 'age',  'oversampling'),
    ('DIATREND',          df_orig_diatrend,  pat_info_diatrend,
     'windows_with_5folds_DiaTrend_2026-03-27_PH4_fold1_sex_undersampling.
↪parquet',              1, 'sex',  'undersampling'),
    ('DIATREND',          df_orig_diatrend,  pat_info_diatrend,
     'windows_with_5folds_DiaTrend_2026-03-27_PH4_fold4_age_smote.parquet',
↪                        4, 'age',  'smote'),
    ('REPLACE-BG',        df_orig_replace,   pat_info_replace,
     'windows_with_5folds_REPLACE-BG_2026-05-26_PH4_fold0_age_reference_proportional.
↪parquet', 0, 'age', 'reference_proportional'),
    ('REPLACE-BG',        df_orig_replace,   pat_info_replace,
     'windows_with_5folds_REPLACE-BG_2026-05-26_PH4_fold1_sex_patient_aware_undersampling.
↪parquet', 1, 'sex', 'patient_aware_undersampling'),
```

```

    ('REPLACE-BG', df_orig_replace, pat_info_replace,
     'windows_with_5folds_REPLACE-BG_2026-05-26_PH4_fold2_age_undersampling.
↳parquet', 2, 'age', 'undersampling'),
    ('T1DiabetesGranada', df_orig_t1dg, pat_info_t1dg,
     'windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4_fold4_age_smote.
↳parquet', 4, 'age', 'smote'),
    ('T1DiabetesGranada', df_orig_t1dg, pat_info_t1dg,
     □
↳'windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4_fold4_sex_oversampling.
↳parquet', 4, 'sex', 'oversampling'),
    ('T1DiabetesGranada', df_orig_t1dg, pat_info_t1dg,
     □
↳'windows_with_5folds_T1DiabetesGranada_2026-05-26_PH4_fold4_sex_undersampling.
↳parquet', 4, 'sex', 'undersampling'),
]

OVERSAMPLE_TECHNIQUES = {'oversampling', 'smote', 'jittering', □
↳'reference_proportional'}
UNDERSAMPLE_TECHNIQUES = {'undersampling', 'patient_aware_undersampling'}

rows = []
for ds_name, df_orig, pat_info, fname, fold_idx, group, technique in □
↳EXPERIMENTS:
    ds_dir = DATASETS[ds_name]
    bal_file = ds_dir / 'balanced_outputs' / fname
    if not bal_file.exists():
        continue

    df_bal = pd.read_parquet(bal_file)
    bal_train = df_bal[df_bal['split'] == 'train'].reset_index(drop=True)
    bal_val = df_bal[df_bal['split'] == 'val'].reset_index(drop=True)
    bal_test = df_bal[df_bal['split'] == 'test'].reset_index(drop=True)
    orig_train = get_original_split(df_orig, fold_idx, 'train')
    orig_val = get_original_split(df_orig, fold_idx, 'val')
    orig_test = get_original_split(df_orig, fold_idx, 'test')

    delta = len(bal_train) - len(orig_train)
    n_dup = count_exact_duplicates(bal_train)

    # Integridad val/test
    val_ok = check_val_test_integrity(orig_val, bal_val, 'val').
↳startswith(' ')
    test_ok = check_val_test_integrity(orig_test, bal_test, 'test').
↳startswith(' ')

    # Solapamiento

```



```

        overlap_ok = check_no_patient_overlap(bal_train, bal_val, bal_test).
        ↪startswith(' ')

    # Efecto esperado en tamaño
    if technique in OVERSAMPLE_TECHNIQUES:
        size_ok = ' ' if delta >= 0 else ' '
    elif technique in UNDERSAMPLE_TECHNIQUES:
        size_ok = ' ' if delta <= 0 else ' '
    else:
        size_ok = '-'

    rows.append({
        'Dataset': ds_name,
        'Fold': fold_idx,
        'Grupo': group,
        'Técnica': technique,
        'Orig. train': f'{len(orig_train):,}',
        'Bal. train': f'{len(bal_train):,}',
        'Δ filas': f'{delta:+,}',
        'Δ OK': size_ok,
        'Dup. train': n_dup,
        'val intacto': ' ' if val_ok else ' ',
        'test intacto': ' ' if test_ok else ' ',
        'Sin overlap': ' ' if overlap_ok else ' ',
    })

summary_df = pd.DataFrame(rows)
display(summary_df.set_index(['Dataset', 'Fold', 'Grupo', 'Técnica']))

```

Dataset	Fold	Grupo	Técnica	Orig. train \
DIATREND	0	age	oversampling	1,598,896
	1	sex	undersampling	1,613,241
	4	age	smote	1,604,177
REPLACE-BG	0	age	reference_proportional	2,308,739
	1	sex	patient_aware_undersampling	2,309,304
	2	age	undersampling	2,313,695
T1DiabetesGranada	4	age	smote	13,583,812
		sex	oversampling	13,583,812
			undersampling	13,583,812

Dataset	Fold	Grupo	Técnica	Bal. train \
DIATREND	0	age	oversampling	6,056,508
	1	sex	undersampling	484,770
	4	age	smote	4,652,544
REPLACE-BG	0	age	reference_proportional	2,308,739

	1	sex	patient_aware_undersampling	2,267,201
	2	age	undersampling	1,277,544
T1DiabetesGranada	4	age	smote	21,499,470
		sex	oversampling	14,818,976
			undersampling	12,348,648

Δ filas Δ OK \

Dataset	Fold	Grupo	Técnica	
DIATREND	0	age	oversampling	+4,457,612
	1	sex	undersampling	-1,128,471
	4	age	smote	+3,048,367
REPLACE-BG	0	age	reference_proportional	+0
	1	sex	patient_aware_undersampling	-42,103
	2	age	undersampling	-1,036,151
T1DiabetesGranada	4	age	smote	+7,915,658
		sex	oversampling	+1,235,164
			undersampling	-1,235,164

Dup. train \

Dataset	Fold	Grupo	Técnica	
DIATREND	0	age	oversampling	4459496
	1	sex	undersampling	319
	4	age	smote	1496
REPLACE-BG	0	age	reference_proportional	273802
	1	sex	patient_aware_undersampling	900
	2	age	undersampling	405
T1DiabetesGranada	4	age	smote	25304
		sex	oversampling	3103782
			undersampling	10558

val intacto \

Dataset	Fold	Grupo	Técnica
DIATREND	0	age	oversampling
	1	sex	undersampling
	4	age	smote
REPLACE-BG	0	age	reference_proportional
	1	sex	patient_aware_undersampling
	2	age	undersampling
T1DiabetesGranada	4	age	smote
		sex	oversampling
			undersampling

test intacto \

Dataset	Fold	Grupo	Técnica
DIATREND	0	age	oversampling
	1	sex	undersampling
	4	age	smote
REPLACE-BG	0	age	reference_proportional

	1	sex	patient_aware_undersampling
	2	age	undersampling
T1DiabetesGranada	4	age	smote
		sex	oversampling
			undersampling

Sin overlap

Dataset	Fold	Grupo	Técnica
DIATREND	0	age	oversampling
	1	sex	undersampling
	4	age	smote
REPLACE-BG	0	age	reference_proportional
	1	sex	patient_aware_undersampling
	2	age	undersampling
T1DiabetesGranada	4	age	smote
		sex	oversampling
			undersampling

1.4.1 Verificar si los duplicados en undersampling son preexistentes

Añadir tras cada sección de undersampling (o como celda genérica al final). La idea es contar duplicados en el train original del mismo fold y comparar con los del balanceado. Si el número es igual o mayor en el original, el undersampling no introdujo ninguno.

```
[16]: # ANÁLISIS: ¿Los duplicados en undersampling son preexistentes?
# Ejecutar para cada caso de undersampling. Cambiar df_orig, fold_idx según
# corresponda.

def check_preexisting_duplicates(df_orig, fold_idx, label=''):
    orig_train = get_original_split(df_orig, fold_idx, 'train')
    n_dup_orig = count_exact_duplicates(orig_train)
    print(f'{label}')
    print(f' Duplicados en train ORIGINAL (fold {fold_idx}): {n_dup_orig:,}')
    return n_dup_orig

print('=== Duplicados preexistentes en train original ===\n')

# DIATREND fold1 · sex · undersampling
n = check_preexisting_duplicates(df_orig_diatrend, 1, 'DIATREND_
↳fold1·sex·undersampling')
print(f' Duplicados en train BALANCEADO : 319')
print(f' Conclusión: {" preexistentes (undersampling no creó nuevos)" if n >=
↳319 else " undersampling introdujo duplicados nuevos - revisar"}\n')

# REPLACE-BG fold2 · age · undersampling
n = check_preexisting_duplicates(df_orig_replace, 2, 'REPLACE-BG_
↳fold2·age·undersampling')
print(f' Duplicados en train BALANCEADO : 405')
```

```

print(f' Conclusión: {" preexistentes" if n >= 405 else " nuevos duplicados -
revisar"}\n')

# REPLACE-BG fold1 · sex · patient_aware_undersampling
n = check_preexisting_duplicates(df_orig_replace, 1, 'REPLACE-BG_
↳fold1·sex·patient_aware_undersampling')
print(f' Duplicados en train BALANCEADO : 900')
print(f' Conclusión: {" preexistentes" if n >= 900 else " nuevos duplicados -
revisar"}\n')

# T1DiabetesGranada fold4 · sex · undersampling
n = check_preexisting_duplicates(df_orig_t1dg, 4, 'T1DiabetesGranada_
↳fold4·sex·undersampling')
print(f' Duplicados en train BALANCEADO : 10,558')
print(f' Conclusión: {" preexistentes" if n >= 10558 else " nuevos_
↳duplicados - revisar"}\n')

```

=== Duplicados preexistentes en train original ===

```

DIATREND fold1·sex·undersampling
  Duplicados en train ORIGINAL (fold 1): 1,619
  Duplicados en train BALANCEADO : 319
  Conclusión: preexistentes (undersampling no creó nuevos)

```

```

REPLACE-BG fold2·age·undersampling
  Duplicados en train ORIGINAL (fold 2): 958
  Duplicados en train BALANCEADO : 405
  Conclusión: preexistentes

```

```

REPLACE-BG fold1·sex·patient_aware_undersampling
  Duplicados en train ORIGINAL (fold 1): 916
  Duplicados en train BALANCEADO : 900
  Conclusión: preexistentes

```

```

T1DiabetesGranada fold4·sex·undersampling
  Duplicados en train ORIGINAL (fold 4): 11,614
  Duplicados en train BALANCEADO : 10,558
  Conclusión: preexistentes

```

1.4.2 Ratio de expansión y severidad del desbalanceo original

Para contextualizar el impacto del balanceo, especialmente en los casos de oversampling masivo de DIATREND

```

[17]: # ANÁLISIS: Severidad del desbalanceo original y ratio de expansión
print('=== Severidad del desbalanceo en train original (ratio max/min) ===\n')

```

```

cases = [
    ('DIATREND', df_orig_diatrend, pat_info_diatrend, 0, 'age',
↪ 'oversampling', 6_056_508),
    ('DIATREND', df_orig_diatrend, pat_info_diatrend, 1, 'sex',
↪ 'undersampling', 484_770),
    ('DIATREND', df_orig_diatrend, pat_info_diatrend, 4, 'age',
↪ 'smote', 4_652_544),
    ('REPLACE-BG', df_orig_replace, pat_info_replace, 0, 'age',
↪ 'reference_proportional', 2_308_739),
    ('REPLACE-BG', df_orig_replace, pat_info_replace, 1, 'sex',
↪ 'patient_aware_undersampling', 2_267_201),
    ('REPLACE-BG', df_orig_replace, pat_info_replace, 2, 'age',
↪ 'undersampling', 1_277_544),
    ('T1DiabetesGranada', df_orig_t1dg, pat_info_t1dg, 4, 'age',
↪ 'smote', 21_499_470),
    ('T1DiabetesGranada', df_orig_t1dg, pat_info_t1dg, 4, 'sex',
↪ 'oversampling', 14_818_976),
    ('T1DiabetesGranada', df_orig_t1dg, pat_info_t1dg, 4, 'sex',
↪ 'undersampling', 12_348_648),
]

rows = []
for ds_name, df_orig, pat_info, fold_idx, group, technique, bal_size in cases:
    orig_train = get_original_split(df_orig, fold_idx, 'train')
    add_fn = add_age_group if group == 'age' else add_sex_group
    orig_g = add_fn(orig_train, 'patient_id', pat_info)
    group_col = f'{group}_group'
    counts = orig_g[group_col].value_counts()
    ratio = counts.max() / counts.min() if counts.min() > 0 else
↪ float('inf')
    rows.append({
        'Dataset': ds_name,
        'Fold': fold_idx,
        'Grupo': group,
        'Técnica': technique,
        'n grupos': len(counts),
        'max grupo': f'{counts.max():,}',
        'min grupo': f'{counts.min():,}',
        'ratio max/min': f'{ratio:.1f}×',
        'train orig': f'{len(orig_train):,}',
        'train bal': f'{bal_size:,}',
        'expansión': f'{bal_size/len(orig_train):.2f}×',
    })

display(pd.DataFrame(rows).set_index(['Dataset', 'Fold', 'Grupo', 'Técnica']))

```

=== Severidad del desbalanceo en train original (ratio max/min) ===

Dataset	Fold	Grupo	Técnica	n grupos	max grupo \
DIATREND	0	age	oversampling	4	1,514,127
	1	sex	undersampling	2	1,370,856
	4	age	smote	3	1,550,848
REPLACE-BG	0	age	reference_proportional	4	791,087
	1	sex	patient_aware_undersampling	2	1,157,535
	2	age	undersampling	4	844,561
T1DiabetesGranada	4	age	smote	4	4,511,256
		sex	oversampling	2	7,409,488
			undersampling	2	7,409,488

Dataset	Fold	Grupo	Técnica	min grupo \
DIATREND	0	age	oversampling	15,900
	1	sex	undersampling	242,385
	4	age	smote	25,115
REPLACE-BG	0	age	reference_proportional	311,392
	1	sex	patient_aware_undersampling	1,151,769
	2	age	undersampling	319,386
T1DiabetesGranada	4	age	smote	2,564,116
		sex	oversampling	6,174,324
			undersampling	6,174,324

Dataset	Fold	Grupo	Técnica	ratio max/min \
DIATREND	0	age	oversampling	95.2×
	1	sex	undersampling	5.7×
	4	age	smote	61.7×
REPLACE-BG	0	age	reference_proportional	2.5×
	1	sex	patient_aware_undersampling	1.0×
	2	age	undersampling	2.6×
T1DiabetesGranada	4	age	smote	1.8×
		sex	oversampling	1.2×
			undersampling	1.2×

Dataset	Fold	Grupo	Técnica	train orig \
DIATREND	0	age	oversampling	1,598,896
	1	sex	undersampling	1,613,241
	4	age	smote	1,604,177
REPLACE-BG	0	age	reference_proportional	2,308,739
	1	sex	patient_aware_undersampling	2,309,304
	2	age	undersampling	2,313,695
T1DiabetesGranada	4	age	smote	13,583,812

		sex	oversampling	13,583,812	
			undersampling	13,583,812	
					train bal expansión
Dataset	Fold	Grupo	Técnica		
DIATREND	0	age	oversampling	6,056,508	3.79×
	1	sex	undersampling	484,770	0.30×
	4	age	smote	4,652,544	2.90×
REPLACE-BG	0	age	reference_proportional	2,308,739	1.00×
	1	sex	patient_aware_undersampling	2,267,201	0.98×
	2	age	undersampling	1,277,544	0.55×
T1DiabetesGranada	4	age	smote	21,499,470	1.58×
		sex	oversampling	14,818,976	1.09×
			undersampling	12,348,648	0.91×

1.4.3 Comprobar que val y test son idénticos entre técnicas del mismo (dataset, fold)

Si dos técnicas distintas actúan sobre el mismo fold, sus val y test deben ser exactamente iguales (porque provienen del mismo fold del original). Ya lo hiciste para T1DG fold4 oversampling vs undersampling, pero vale la pena hacerlo sistemáticamente para todos los casos donde hay más de un archivo del mismo fold:

```
[18]: # ANÁLISIS: val/test consistentes entre técnicas del mismo fold
# Agrupar archivos disponibles por (dataset, fold)
from collections import defaultdict

by_fold = defaultdict(list)
all_bal_files = []
for ds_name, ds_dir_ in DATASETS.items():
    bo = ds_dir_ / 'balanced_outputs'
    if bo.exists():
        for f in sorted(bo.glob('windows_with_5folds_*.parquet')):
            m = BAL_PATTERN.match(f.name)
            if m:
                all_bal_files.append((ds_name, int(m.group('fold')), f))
                by_fold[(ds_name, int(m.group('fold')))].append(f)

print('=== Consistencia de val/test entre técnicas del mismo (dataset, fold) _
↳===\n')

for (ds_name, fold_idx), files in sorted(by_fold.items()):
    if len(files) < 2:
        continue # solo 1 archivo para este fold, nada que comparar
    print(f' {ds_name} - fold {fold_idx} ({len(files)} técnicas):')
    # Tomar el primero como referencia
    ref_df = pd.read_parquet(files[0])
    ref_val = ref_df[ref_df['split'] == 'val'].reset_index(drop=True)
    ref_test = ref_df[ref_df['split'] == 'test'].reset_index(drop=True)
    for f in files[1:]:
```

```

cmp_df    = pd.read_parquet(f)
cmp_val   = cmp_df[cmp_df['split'] == 'val'].reset_index(drop=True)
cmp_test  = cmp_df[cmp_df['split'] == 'test'].reset_index(drop=True)
rv = check_val_test_integrity(ref_val, cmp_val, 'val').split(':')[0]
rt = check_val_test_integrity(ref_test, cmp_test, 'test').split(':')[0]
print(f'    {files[0].name[-40:]}')
print(f'    vs {f.name[-40:]}')
print(f'    val: {rv} | test: {rt}')
print()

```

=== Consistencia de val/test entre técnicas del mismo (dataset, fold) ===

```

T1DiabetesGranada - fold 4 (3 técnicas):
a_2026-05-26_PH4_fold4_age_smote.parquet
vs 05-26_PH4_fold4_sex_oversampling.parquet
val:  val | test:  test
a_2026-05-26_PH4_fold4_age_smote.parquet
vs 5-26_PH4_fold4_sex_undersampling.parquet
val:  val | test:  test

```

1.5 Conclusiones de la Validación del Balanceo Demográfico

1.5.1 Objetivo de esta validación

Antes de utilizar los datasets balanceados en el pipeline de entrenamiento LSTM, es imprescindible verificar que el proceso de balanceo ha sido implementado correctamente desde el punto de vista metodológico. En concreto, se han comprobado cuatro propiedades que son condición necesaria para que los resultados experimentales sean válidos:

1. **Efecto correcto sobre train:** cada técnica debe producir el cambio de volumen esperado (aumento en oversampling/SMOTE/jittering, reducción en undersampling, conservación en reference_proportional).
2. **Inmutabilidad de val y test:** los conjuntos de evaluación de cada fold no deben haber sido tocados en ningún caso.
3. **Ausencia de solapamiento de pacientes:** ningún paciente puede aparecer simultáneamente en train, val y test dentro del mismo fold.
4. **Consistencia de val/test entre técnicas:** dos técnicas distintas aplicadas sobre el mismo fold deben producir exactamente los mismos conjuntos de val y test, ya que ambos provienen del mismo fold del archivo original.

Las tres primeras garantías son condición suficiente para que el balanceo sea correcto a nivel de fold. La cuarta garantiza que la comparación entre técnicas es justa.

1.5.2 Resultado 1 — Efecto correcto del balanceo sobre train (todos los casos)

Dataset	Fold	Grupo	Técnica	Δ filas	Resultado
DIATREND	0	age	oversampling	+4.457.612	
DIATREND	1	sex	undersampling	−1.128.471	
DIATREND	4	age	smote	+3.048.367	
REPLACE-BG	0	age	reference_proportional	0	
REPLACE-BG	1	sex	patient_aware_undersampling	−42.103	
REPLACE-BG	2	age	undersampling	−1.036.151	
T1DiabetesGranada	4	age	smote	+7.915.658	
T1DiabetesGranada	4	sex	oversampling	+1.235.164	
T1DiabetesGranada	4	sex	undersampling	−1.235.164	

Todos los experimentos producen el cambio de volumen correcto según la técnica aplicada. El caso de `reference_proportional` en REPLACE-BG es particularmente ilustrativo: el delta es exactamente 0, lo que confirma que la técnica redistribuye las ventanas entre grupos (internamente hace oversampling de los grupos por debajo del objetivo y undersampling de los que están por encima) sin alterar el volumen total de entrenamiento. Esto es el comportamiento diseñado.

1.5.3 Resultado 2 — Inmutabilidad de val y test (todos los casos)

En los 9 experimentos, los conjuntos `val` y `test` de cada fold son bit-a-bit idénticos a los del archivo original filtrado al mismo fold. Esto es la garantía fundamental de ausencia de data leakage: el balanceo opera exclusivamente sobre las ventanas de entrenamiento y no tiene acceso —ni indirecto— a las ventanas de evaluación.

La metodología que lo hace posible es la siguiente: el script `2b-trainset_balancing.py` separa las filas de `val` y `test` *antes* de ejecutar cualquier técnica de balanceo, las almacena en un dataframe aparte que no participa en ningún cálculo estadístico, y las reincorpora al archivo de salida únicamente al final, tras finalizar el balanceo de train. Ningún estadístico (media, desviación estándar, vecinos más cercanos en SMOTE) se calcula sobre datos de val o test.

1.5.4 Resultado 3 — Ausencia de solapamiento de pacientes (todos los casos)

En los 9 experimentos, los conjuntos train, val y test no comparten ningún `patient_id`. Esto confirma que el esquema de Group K-Fold implementado en la etapa de generación de ventanas se preserva intacto a través del proceso de balanceo.

Este punto es especialmente relevante para el oversampling y SMOTE: al replicar o interpolar ventanas de train, se podría haber introducido accidentalmente ventanas de un paciente que en ese fold pertenecía a val o test. La verificación confirma que esto no ha ocurrido, porque el balanceo opera sobre subconjuntos de `patient_id` que ya estaban asignados exclusivamente a train.

1.5.5 Resultado 4 — Consistencia de val/test entre técnicas del mismo fold ()

Para T1DiabetesGranada fold 4, donde están disponibles tres técnicas distintas (`age·smote`, `sex·oversampling`, `sex·undersampling`), se ha verificado que los tres archivos producen exactamente los mismos conjuntos de val y test. Esto confirma que la comparación entre técnicas es metodológicamente justa: todas se evalúan sobre los mismos pacientes y las mismas ventanas temporales, y cualquier diferencia en las métricas finales (RMSE, TIR, TAR, TBR) es atribuible exclusivamente al efecto del balanceo sobre el entrenamiento.

1.5.6 Resultado 5 — Los duplicados en undersampling son preexistentes ()

Se observaron duplicados residuales en los cuatro casos de técnicas reductoras:

Experimento	Dup. en orig. train	Dup. en bal. train
DIATREND fold1 · sex · undersampling	1.619	319
REPLACE-BG fold2 · age · undersampling	958	405
REPLACE-BG fold1 · sex · patient_aware_undersampling	916	900
T1DiabetesGranada fold4 · sex · undersampling	11.614	10.558

En los cuatro casos, el número de duplicados en el dataset balanceado es **menor o igual** que en el original. Esto descarta que el undersampling haya introducido duplicados nuevos: simplemente ha conservado un subconjunto de los que ya existían en el train original, porque el muestreo aleatorio sin reemplazo no tiene mecanismo para crear duplicados.

La presencia de duplicados preexistentes en los datasets originales es un fenómeno esperable en series CGM de larga duración: cuando un paciente mantiene su glucosa estable durante un período prolongado, ventanas contiguas pueden producir vectores ($\mathbf{x}_0, \dots, \mathbf{x}_7, \mathbf{y}$) numéricamente idénticos. No representa un problema para el entrenamiento, ya que su proporción es marginal ($< 0,1\%$ en todos los casos).

1.5.7 Resultado 6 — Severidad del desbalanceo original y racionalidad de las expansiones

El análisis del ratio `max_grupo / min_grupo` en el train original revela diferencias importantes entre datasets y dimensiones:

DIATREND por edad es el caso más extremo: ratio de $95,2\times$ en fold 0 y $61,7\times$ en fold 4. En la práctica, esto significa que el grupo mayoritario (adultos de mediana edad) tiene entre 60 y 95 veces más ventanas que el grupo minoritario (adolescentes o mayores de 60 años). Esta asimetría es el origen de las expansiones de $3,79\times$ y $2,90\times$ observadas en oversampling y SMOTE respectivamente. En el caso del oversampling, el grupo minoritario se replica ~ 95 veces para igualar al mayoritario, lo que implica que prácticamente todas las ventanas nuevas son copias de las mismas pocas ventanas originales. Por este motivo, en presencia de ratios tan extremos, **SMOTE es metodológicamente superior al oversampling aleatorio**: genera interpolaciones distintas en lugar de réplicas exactas, aportando mayor diversidad sintética.

DIATREND por sexo muestra un ratio de $5,7\times$, moderado. El undersampling descarta el 70% del volumen de train, lo que es una pérdida importante pero manejable.

REPLACE-BG presenta el desbalanceo más moderado en todos los casos: ratio $2,5\times$ por edad y prácticamente $1,0\times$ por sexo (fold 1), donde la diferencia entre grupos es tan pequeña que el patient-aware undersampling apenas reduce el tamaño ($-1,8\%$). Este resultado indica que REPLACE-BG está ya razonablemente equilibrado por sexo, y que el balanceo en esa dimensión puede tener un impacto experimental mínimo.

T1DiabetesGranada muestra ratios moderados tanto por edad ($1,8\times$) como por sexo ($1,2\times$). La expansión de SMOTE por edad ($1,58\times$) y del oversampling por sexo ($1,09\times$) son las más contenidas del experimento, lo que sugiere que T1DG es el dataset con mayor equidad demográfica de partida.

1.5.8 Validación global: resumen ejecutivo

Todas las propiedades necesarias para garantizar la validez metodológica del balanceo han sido verificadas y confirmadas en los 9 experimentos analizados:

Propiedad verificada	Resultado
Efecto correcto sobre train ($\uparrow$ / $\downarrow$ / $=$ según técnica)	9/9
val intacto (bit-a-bit idéntico al original)	9/9
test intacto (bit-a-bit idéntico al original)	9/9
Sin solapamiento de pacientes entre splits	9/9
val/test consistentes entre técnicas del mismo fold	3/3 (T1DG fold 4)
Duplicados en undersampling son preexistentes	4/4

Los datasets balanceados son metodológicamente correctos y pueden ser utilizados directamente como entrada del pipeline de entrenamiento LSTM. Las comparaciones entre la condición original y cada técnica de balanceo son justas, ya que todas se evalúan sobre exactamente los mismos conjuntos de validación y test.